

经典机器学习算法详解

经典机器学习算法是现代深度学习的基石。理解这些算法的数学原理、适用场景与工程实践，是构建可靠 AI 系统的前提。本文系统梳理监督学习、无监督学习与集成学习的核心算法，并给出基于 Scikit-learn 的实践示例。

经典机器学习算法详解

经典机器学习算法是现代深度学习的基石。理解这些算法的数学原理、适用场景与工程实践，是构建可靠 AI 系统的前提。本文系统梳理监督学习、无监督学习与集成学习的核心算法，并给出基于 Scikit-learn 的实践示例。

- - -

一、监督学习算法

监督学习 (Supervised Learning) 的训练数据形如 $D = \{(x_i, y_i)\}$ ，其中 x_i 为特征向量， y_i 为标签。根据 y 是连续值还是离散值，可分为回归 (Regression) 与分类 (Classification) 两类任务。

1.1 线性回归与逻辑回归

线性回归 (Linear Regression)

模型假设输出与特征之间存在线性关系：

$$\hat{y} = w^T x + b$$

通过最小化均方误差 (MSE) 求解参数：

$$\min_{w, b} \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

加入 L_2 正则化项后称为岭回归 (Ridge)，加入 L_1 正则化项称为 Lasso 回归，可产生稀疏解。解析解 (闭式解) 为：

$$w^* = (X^T X + \lambda I)^{-1} X^T y$$

逻辑回归 (Logistic Regression)

虽名为回归，实为二分类算法。通过 sigmoid 函数将线性输出映射到概率：

$$p(y=1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

使用交叉熵损失训练：

$$L = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \right]$$

特点：可解释性强、训练快、可作为概率输出；但只能处理线性可分问题，需通过特征交叉或核技巧扩展。

1.2 决策树与随机森林

决策树 (Decision Tree)

通过递归划分特征空间构建树结构。每个内部节点选择一个特征及阈值进行分裂，叶节点给出预测值。分裂准则

- 信息增益 (ID3) : $IG(D, A) = H(D) - H(D|A)$ ，使用熵 $H(D)$
- 信息增益率 (C4.5) : 修正信息增益对多值特征的偏好
- 基尼系数 (CART) : $Gini(D) = 1 - \sum p_k^2$ ，计算更高效

随机森林 (Random Forest) 是 Bagging 的代表算法，构建流程：

1. 从训练集中有放回采样 (Bootstrap) 得到 T 个子集
2. 对每个子集训练一棵决策树，每次分裂时仅随机选取部分特征
3. 预测时取所有树结果的平均 (回归) 或多数投票 (分类)

优势：抗过拟合、能评估特征重要性、并行训练；劣势：模型体积大、外推能力弱、可解释性弱于单棵树。

1.3 支持向量机 (SVM)

SVM 寻找使类别间隔最大化的超平面：

$$\max_{w, b} \frac{1}{\|w\|} \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1$$

转化为对偶问题后引入核函数 $K(x_i, x_j)$ 处理非线性：

核函数 表达式 适用场景

线性核 $K(x, y) = x^T y$ 高维稀疏特征 (文本)

多项式核 $K(x, y) = (x^T y + c)^d$ 中等维度非线性

RBF 核 $K(x, y) = \exp(-\gamma \|x - y\|^2)$ 通用、最常用

Sigmoid 核 $K(x, y) = \tanh(\alpha x^T y + c)$ 类似浅层神经网络

关键超参数：惩罚系数 C （越大越倾向硬间隔）、核参数 γ （越大支持向量影响越局部）
在小样本、高维场景下表现优异，但训练复杂度 $O(N^2)$ 至 $O(N^3)$ ，不适合大规模数据。

1.4 朴素贝叶斯

基于贝叶斯定理与特征条件独立假设：

$$P(y|x) = \frac{P(y) \prod_j P(x_j|y)}{P(x)}$$

常见变体：

- 高斯朴素贝叶斯：假设连续特征服从高斯分布
- 多项式朴素贝叶斯：适用于文本分类的词频特征
- 伯努利朴素贝叶斯：适用于二值特征

优点：训练极快、对小数据稳健、天然支持多分类；缺点：条件独立假设过强，特征相关时性能下降。

1.5 K近邻 (KNN)

KNN 是 "懒学习" 代表，没有显式训练过程。预测时找到距 x 最近的 K 个训练样本，由它们标签的多数投票（分类）或平均（回归）决定结果。

距离度量包括欧氏距离、曼哈顿距离、余弦相似度等。K 值选择通过交叉验证确定：K 太小易过拟合，太大欠拟合。可使用 KD-Tree 或 Ball-Tree 加速近邻搜索，将复杂度从 $O(N \cdot d)$ 降至 $O(\log N \cdot d)$ 。

- - -

二、无监督学习算法

无监督学习的训练数据仅有 $D = \{x_i\}_{i=1}^N$ ，目标是发现数据的内在结构。

2.1 K-Means 聚类

目标：将样本划分为 K 个簇，使簇内方差之和最小：

$$\min_C, \mu \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2$$

Lloyd 算法迭代步骤：

1. 随机初始化 K 个簇中心 μ_k

- 2 . 分配步：将每个样本分配到最近的簇中心
- 3 . 更新步：重新计算每个簇的样本均值作为新中心
- 4 . 重复 2 - 3 直到中心不再变化

K - M e a n s ++ 初始化：使初始中心彼此距离较远，显著降低陷入局部最优的概率。缺点：需预设 K、对异常值敏感、只适合凸形簇、对初始中心敏感。

2 . 2 层次聚类

构建层次化的聚类树（D e n d r o g r a m），分为两种策略：

- 凝聚式（A g g l o m e r a t i v e，自底向上）：每个样本初始为一簇，逐步合并最相似的簇
- 分裂式（D i v i s i v e，自顶向下）：所有样本初始为一簇，逐步分裂

簇间距离的链接方式（L i n k a g e）决定合并行为：

L i n k a g e 定义 特点

S i n g l e 两簇最近点距离 易产生链式合并

C o m p l e t e 两簇最远点距离 偏好紧凑球形簇

A v e r a g e 两簇平均距离 折中选择

W a r d 合并后方差增量最小 类似 K - M e a n s 目标

优点：不需预设簇数、可视化清晰；缺点：复杂度 $O(N^3)$ 、不可撤销合并。

2 . 3 D B S C A N

基于密度的聚类，能发现任意形状的簇并识别噪声点。核心概念：

- 核心点： ϵ 邻域内样本数 $\geq MinPts$
- 边界点：在核心点邻域内但自身非核心点
- 噪声点：既非核心点也非边界点

算法流程：从任一未访问核心点出发，密度可达的所有点构成一个簇。

优点：不需指定簇数、能识别噪声、可处理非凸簇；缺点：对 ϵ 与 $MinPts$ 敏感、对密度变化大的数据效果差、高维下距离度量失效。H D B S C A N 是其改进版本，支持变密度聚类。

2 . 4 P C A 降维

主成分分析（Principal Component Analysis）通过线性变换将数据投影到方差最大的正交基上。

- 1. 中心化数据： $X' = X - \bar{X}$
- 2. 计算协方差矩阵： $\Sigma = \frac{1}{N} X'^T X'$
- 3. 特征值分解： $\Sigma = U \Lambda U^T$
- 4. 取前 k 个最大特征值对应的特征向量组成投影矩阵 W_k
- 5. 降维结果： $Z = X' W_k$

数学解释：PCA 等价于在最小重构误差意义下寻找最优线性子空间。

2.5 t-SNE 与 UMAP

两者均为非线性降维方法，主要用于高维数据可视化。

t-SNE：高维空间用高斯分布衡量相似度，低维空间用学生 t 分布（自由度 1），通过 KL 散度优化。保留局部结构效果好，但计算慢、全局结构失真。

UMAP：基于黎曼几何与代数拓扑，构造高维与低维的模糊拓扑图并最小化交叉熵。比 t-SNE 更快、保留全局结构更好、可增量学习。

维度 PCA t-SNE UMAP

类型 线性 非线性 非线性

主要用途 降维 / 去噪 可视化 可视化 / 降维

保留全局结构 强 弱 较强

训练速度 极快 慢 较快

可逆性 是 否 否

- - -

三、集成学习（Bagging、Boosting、Stacking）

集成学习通过组合多个弱学习器获得更强的泛化性能。

3.1 Bagging

Bootstrap Aggregating：并行训练多个基学习器，各自基于 Bootstrap 采样得到的数据子集，最终通过投票或平均聚合。代表算法：随机森林。主要降低方差，适合高方差模型（如深度学习）。

3.2 Boosting

串行训练：每个新模型专注于纠正前序模型的错误。主要降低偏差。

- AdaBoost：增加错分样本权重，加权组合弱分类器
- GBDT：以负梯度作为新模型的拟合目标，使用决策树为基学习器
- XGBoost：引入二阶泰勒展开与正则化，支持并行与缺失值处理
- LightGBM：基于直方图与 Leaf-wise 生长，速度快、内存省
- CatBoost：原生处理类别特征，使用 Ordered Boosting 避免预测偏移

3.3 Stacking

训练多个异构基学习器，将其输出作为新特征训练元学习器（Meta-learner）。为避免数据泄露，K 折交叉验证生成。

3.4 三类集成对比

维度 Bagging Boosting Stacking

训练方式 并行 串行 两阶段

目标 降低方差 降低偏差 综合提升

基学习器 高方差模型 高偏差模型 异构模型

代表算法 随机森林 XGBoost / LightGBM —

— — —

四、算法选型决策表

数据特征 / 任务需求 推荐算法 备选算法

小样本、高维线性分类 逻辑回归 / 线性 SVM 朴素贝叶斯

小样本、非线性分类 RBF-SVM 随机森林

中等规模表格数据 LightGBM / XGBoost 随机森林

大规模稀疏文本分类 逻辑回归 + TF-IDF 朴素贝叶斯

可解释性要求高 决策树 / 逻辑回归 GAM

凸形簇聚类 K-Means GMM

任意形状簇 + 噪声 DBSCAN / HDBSCAN 谱聚类

高维数据可视化 UMAP t - SNE

线性降维去噪 PCA 截断 SVD

- - -

五、Scikit-learn 实践示例

```
import numpy as np
from sklearn.datasets import make_classification,
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.metrics import classification_report

- - - - 分类任务对比 - - - -
X, y = make_classification(n_samples=2000, n_features=
n_redundant=5, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X,
stratify=y, random_state=42)

models = {
    "LogReg": Pipeline([("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000))]),
    "SVM-RBF": Pipeline([("scaler", StandardScaler()),
    ("clf", SVC(C=1.0, gamma="scale"))]),
    "RandomForest": RandomForestClassifier(n_estimators=200,
n_jobs=-1, random_state=42),
    "GBDT": GradientBoostingClassifier(n_estimators=200,
max_depth=3, random_state=42),
}
```

```

for name, model in models.items():
    scores = crossvalscore(model, Xtrain, ytrain, cv=5)
    print(f"{name}: F1 = {scores.mean():.4f} ± {scores.std():.4f}")

- - - - 聚类与降维 - - - -

iris = load_iris()
Xiris = StandardScaler().fit_transform(iris.data)

kmeans = KMeans(nclusters=3, init="k-means++", n_init=10)
labelskm = kmeans.fit_predict(Xiris)

dbscan = DBSCAN(eps=0.6, minsamples=5)
labelbdb = dbscan.fit_predict(Xiris)

pca = PCA(ncomponents=2)
Z = pca.fit_transform(Xiris)
print(f"PCA explained variance ratio: {pca.explained_variance_ratio_}")
print(f"KMeans cluster sizes: {np.bincount(labelskm).tolist()}")
print(f"DBSCAN noise points: {(labelbdb == -1).sum()}")

```

工程要点：

1. 数据泄露防范：所有预处理（标准化、PCA 等）必须放入 Pipeline，并在交叉验证内部执行。
2. 类别不平衡：使用 `class_weight='balanced'` 或采样策略（SMOTE）。
3. 超参数调优：优先使用 GridSearchCV 或 Optuna 进行贝叶斯优化。
4. 特征重要性：树模型可输出 `feature_importances_`，配合 SHAP 进行解释。

- - -

六、关键词

机器学习算法、决策树、随机森林、SVM、K-Means、PCA、集成学习、Scikit-learn